

- RAG Learning

4.6 Fundamentals of Computer Systems

Computer Science — AQA A-Level (7517) — Paper 2: Written Theory

Topic 4.6 binds the abstract (logic, languages, algorithms) to the physical (silicon, voltages, clocks). It opens with the hardware/software divide, steps up through language classifications and translators, drops down to Boolean algebra and gate-level circuitry, then climbs back to the operating system that orchestrates it all. This note walks sections 4.6.1–4.6.7 plus the combinatorial, sequential and OS content that AQA examines under this umbrella, in spec order, with the worked truth tables, Karnaugh maps and algebraic proofs that earn method marks.

4.6.1 Hardware and Software

Hardware — the physical components of a computer system: the CPU, RAM, storage devices, motherboard, input/output peripherals, cables and power supply. Anything you can unplug.

Software — the programs and data that run on the hardware. Intangible; distributed as files or transmitted as bit streams. Anything you can *copy*.

Software is further classified into two families — **system software**, which manages the machine itself, and **application software**, which performs tasks useful to the end user.

System software

- **Operating system** — the master control program. Allocates CPU time, memory, files and I/O between processes, and presents a uniform interface to applications. Examples: Windows, macOS, Linux, Android.

- **Utility programs** — small tools that perform housekeeping: disk defragmenter, file compression (ZIP), antivirus, backup, task manager. Usually bundled with the OS.
- **Library programs** — precompiled code modules that applications can call rather than reinvent. Examples: the C standard library, `DirectX`, `libm` for maths functions, Python's `math` and `random` modules.
- **Translators** — assemblers, compilers and interpreters (see 4.6.3). They convert source code into a form the CPU can execute.

Application software

- **General-purpose** — flexible tools designed to handle many kinds of task: Microsoft Word, Excel, Chrome, Photoshop. One product serves millions of users.
- **Special-purpose** — does one specific job very well but nothing else: a chess engine, a tax-return calculator, a supermarket till.
- **Bespoke (custom)** — written to order for a single client. Examples: NHS patient record systems, air-traffic control consoles, a bank's fraud-detection engine.

Bespoke vs off-the-shelf software

	Bespoke (custom)	Off-the-shelf
Cost	High — pays the entire development team.	Low — development cost spread across all customers.
Fit to requirements	Exact; built to the client's workflow.	Approximate; features you need plus many you don't.
Time to deploy	Months or years.	Minutes — install and run.
Maintenance	Depends on one supplier; expensive if they disappear.	Patched by the vendor for all users; large support community.
Reliability at launch	Low — never tested at scale.	High — bugs found and fixed by millions of users.

Competitive advantage	High — no rival has the same tool.	Low — competitors use the same product.
-----------------------	------------------------------------	---

Exam Tip: When an exam question says "a hospital needs patient-booking software", choose *bespoke* and justify with at least *two* of: exact fit to hospital workflow, integration with legacy systems, compliance with NHS data rules, no unnecessary features cluttering the interface. Avoid the vague "it does what you want".

4.6.2 Classification of Programming Languages

Programming languages sit on a spectrum from **low-level** (close to the hardware, one instruction per machine operation) to **high-level** (close to human thought, one statement may expand into dozens of machine instructions).

Low-level languages

- **Machine code** — raw binary instructions the CPU executes directly. Each instruction is a bit pattern matching the processor's instruction set. Not portable: ARM binaries do not run on x86.
- **Assembly language** — mnemonics (`LDR` , `ADD` , `MOV` , `BEQ`) that map one-to-one onto machine instructions. Still processor-specific, but far more readable than hexadecimal. Translated to machine code by an *assembler*.

High-level languages and paradigms

A **paradigm** is a style of programming — a model of how a program is structured. Most modern languages support more than one paradigm, but each has a dominant style.

Paradigm	Core idea	Example languages
Imperative	Sequence of commands that change state. "Do this, then do that."	C, Pascal, early BASIC

Object-oriented (OOP)	Programs are collections of <i>objects</i> that encapsulate data + behaviour and send messages to each other.	Java, C#, Python, C++
Declarative	Describe <i>what</i> the answer is, not <i>how</i> to compute it. Includes logic languages.	SQL, Prolog, HTML
Functional	Computation as the evaluation of mathematical functions; no mutable state, no side-effects.	Haskell, F#, Lisp, Scheme

High-level vs low-level: trade-offs

	High-level	Low-level
Portability	High — compile once per platform.	None — tied to one instruction set.
Readability	Close to English/maths.	Terse; requires knowledge of registers, flags.
Development speed	Fast — rich libraries.	Slow — every detail is manual.
Execution speed	Slower — runtime overhead, abstraction layers.	Fast — programmer controls every cycle.
Hardware access	Indirect via OS calls/libraries.	Direct — can toggle individual register bits.
Typical use	Applications, web, games, business.	Device drivers, embedded firmware, bootloaders, critical kernels.

Exam Tip: "Why write a device driver in assembly?" answers must cite *two* concrete reasons, e.g. *direct access to hardware registers* and *fine-grained timing control*. Do not just say "it's faster" — examiners want the mechanism, not the symptom.

4.6.3 Types of Program Translator

Compiler — a translator that converts an entire high-level source program into equivalent machine (or object) code in a single translation step. The output executable can then be run many times without the compiler being present.

Interpreter — a translator that reads the source program one statement at a time, converts it to machine operations and executes it immediately. The interpreter must be present every time the program runs.

Assembler

Converts assembly language into machine code. Because the mapping is one-to-one (roughly), assembly translation is straightforward: each mnemonic is looked up in an opcode table and the operand fields are encoded beside it. Produces an object file that the linker combines with libraries into an executable.

Compiler vs interpreter

	Compiler	Interpreter
Translation timing	Once, before execution.	Every line, every run.
Output	A standalone executable.	No persistent output; results produced directly.
Execution speed	Fast — translation done.	Slow — re-translates each time through a loop.
Error reporting	All errors reported after full compile.	Stops at the first runtime error encountered.
Development cycle	Edit → compile → link → run. Slower.	Edit → run. Rapid iteration.

Portability of output	Executable tied to one CPU/OS.	Source runs anywhere the interpreter is installed.
Memory at runtime	Only the executable.	Executable <i>and</i> interpreter.
Typical languages	C, C++, Rust, Go.	Python (default), JavaScript, BASIC.

Intermediate code (bytecode) and JIT

A compromise between the two extremes: the compiler produces **intermediate code** (also called bytecode) for a virtual machine rather than a specific CPU. Examples: Java bytecode for the JVM, .NET CIL for the CLR, Python `.pyc` files. The bytecode is then either interpreted by the VM or translated on the fly.

Just-in-time (JIT) compilation profiles the bytecode as it runs and compiles hot loops to native machine code on demand, caching the result for subsequent iterations. Java HotSpot, V8 (Chrome) and .NET RyuJIT are JIT compilers. This gives the portability of bytecode with execution speeds approaching native compilation.

Exam Tip: When AQA asks "compare compilation and interpretation", frame your answer as a table of *at least three* rows: translation timing, execution speed, error reporting. One sentence each. Examiners mark against discriminators, not generalities.

When to use each

- **Interpreter** — during development (fast feedback), for scripting, for platform-independent distribution of source, for interactive shells.
- **Compiler** — for shipping finished products, where runtime speed matters, where source must be concealed, for embedded targets with no room for an interpreter.
- **Bytecode + JIT** — when you need cross-platform deployment *and* near-native speed (enterprise Java, modern JavaScript in browsers).

4.6.4 Logic Gates

Logic gate — a physical electronic component (usually built from transistors) that implements a Boolean function of one or more binary inputs. Inputs and outputs are voltage levels interpreted as 0 (low) or 1 (high).

Truth table — a tabular listing of every possible combination of input values to a Boolean function alongside the corresponding output. An n -input function has 2^n rows.

The six gates AQA examines

Each gate has a standardised symbol. Described in text (the shapes are learned for the exam):

- **AND** — flat-backed shield with a rounded nose. Output 1 only if both inputs are 1.
- **OR** — curved-backed shield with a pointed nose. Output 1 if either input is 1.
- **NOT** — triangle with a small circle (bubble) at the output. Inverts the input.
- **NAND** — AND shape with an output bubble. AND followed by NOT.
- **NOR** — OR shape with an output bubble. OR followed by NOT.
- **XOR** — OR shape with an extra curved arc on the input side. Output 1 iff inputs differ.

Truth tables

A	B	A AND B	A OR B	A XOR B	A NAND B	A NOR B
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

A	NOT A
0	1

1

0

Universal gates: NAND and NOR

A set of gates is **functionally complete** if any Boolean function can be built from it. NAND alone is complete, and so is NOR alone. This means entire CPUs can (and sometimes are) manufactured from a single gate type, which simplifies fabrication.

Building NOT, AND, OR from NAND only:

$$\begin{aligned}\text{NOT } A &= A \text{ NAND } A \\ A \text{ AND } B &= (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B) \\ A \text{ OR } B &= (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)\end{aligned}$$

Verification of A OR B via NAND:

$$\begin{aligned}\text{Let } X &= A \text{ NAND } A = \text{NOT } A \\ \text{Let } Y &= B \text{ NAND } B = \text{NOT } B \\ X \text{ NAND } Y &= \text{NOT}(\text{NOT } A \text{ AND } \text{NOT } B) = A \text{ OR } B \quad (\text{De Morgan})\end{aligned}$$

XOR from NAND (worked example)

XOR needs four NANDs:

$$\begin{aligned}N1 &= A \text{ NAND } B \\ N2 &= A \text{ NAND } N1 \\ N3 &= B \text{ NAND } N1 \\ \text{Out} &= N2 \text{ NAND } N3 = A \text{ XOR } B\end{aligned}$$

Check when A=1, B=0:

$$\begin{aligned}N1 &= 1 \text{ NAND } 0 = 1 \\ N2 &= 1 \text{ NAND } 1 = 0 \\ N3 &= 0 \text{ NAND } 1 = 1 \\ \text{Out} &= 0 \text{ NAND } 1 = 1 \quad (\text{correct: } 1 \text{ XOR } 0 = 1)\end{aligned}$$

Exam Tip: In "build X using only NAND" questions, start by writing X in terms of AND, OR, NOT, then substitute each primitive with its NAND equivalent. Trying to see the NAND circuit directly almost always fails.

4.6.5 Boolean Algebra

Boolean algebra is the algebra of two values (0, 1) with three operations: AND (denoted $\cdot$ or juxtaposition), OR (denoted $+$) and NOT (denoted by an overbar, written here as A'). It lets us simplify circuits *before* committing them to silicon.

Identities and laws

Law	AND form	OR form
Identity	$A \cdot 1 = A$	$A + 0 = A$
Null (annulment)	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot A' = 0$	$A + A' = 1$
Double negation	$(A')' = A$	
Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Absorption	$A \cdot (A + B) = A$	$A + (A \cdot B) = A$
De Morgan's 1st	$(A \cdot B)' = A' + B'$	
De Morgan's 2nd	$(A + B)' = A' \cdot B'$	

De Morgan's laws — proof by truth table

The two laws say that negating an AND gives an OR of negations, and vice versa. Verify the first form exhaustively:

A	B		$A \cdot B$	$(A \cdot B)'$		A'	B'	$A' + B'$
0	0		0	1		1	1	1
0	1		0	1		1	0	1
1	0		0	1		0	1	1
1	1		1	0		0	0	0

Columns $(A \cdot B)'$ and $A' + B'$ match row-for-row. QED.

The second form is proved identically by swapping the middle operator. De Morgan lets us “push” a bubble through a gate, converting an AND-NOT into an OR-of-NOTs — crucial for simplification and for converting between NAND-only and NOR-only implementations.

Worked simplification: $(A \text{ AND } B) \text{ OR } (A \text{ AND NOT } B) = A$

$$\begin{aligned}
 & (A \cdot B) + (A \cdot B') \\
 = & A \cdot (B + B') && [\text{distributive, factor out } A] \\
 = & A \cdot 1 && [\text{complement: } B + B' = 1] \\
 = & A && [\text{identity}]
 \end{aligned}$$

A two-gate circuit (AND, AND, OR) collapses to a wire.

4.6.6 Karnaugh Maps (beyond the AQA 7517 specification)

Karnaugh map (K-map) — a two-dimensional grid representation of a truth table, arranged so that adjacent cells differ in exactly one variable (Gray code ordering). Grouping adjacent 1s visually simplifies a Boolean expression without algebraic manipulation.

Rules for grouping

1. Group *only 1s* (for sum-of-products). Groups of 0s give product-of-sums.
2. Group sizes must be a **power of 2**: 1, 2, 4, 8, 16.
3. Groups must be **rectangular** — no diagonals, no L-shapes.
4. Make each group as **large as possible**; larger groups eliminate more variables.

- Use as **few groups as possible**, but every 1 must be covered by at least one group.
- Groups may **wrap around** edges (top↔bottom, left↔right) because those cells are adjacent in Gray-code terms.
- Groups may **overlap**.

2-variable K-map

	B=0	B=1	
A=0	0	1	cell A=0,B=1
A=1	1	1	cell A=1,B=0 and A=1,B=1

Reading groups: the bottom row (two adjacent 1s) is $A=1$ regardless of $B \rightarrow$ term A . The right column (two adjacent 1s) is $B=1$ regardless of $A \rightarrow$ term B . Expression: $F = A + B$.

3-variable K-map: worked example

Suppose $F(A,B,C) = 1$ for minterms $\{1, 3, 5, 7\}$ (i.e. whenever $C = 1$). The grid uses Gray-coded column labels BC : 00, 01, 11, 10.

A \ BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

The four 1s form one 2×2 group covering the two middle columns. Across the group, A varies (top and bottom rows), B varies (column 01 has $B=0$, column 11 has $B=1$), and only C is constant at 1. Simplified: $F = C$.

3-variable K-map: a second example

$F(A,B,C) = 1$ for minterms $\{0, 2, 4, 6\}$ (the even-indexed minterms — $C = 0$ in each).

A \ BC	00	01	11	10
0	1	0	0	1
1	1	0	0	1

The 1s sit in columns 00 and 10, which wrap around to be adjacent (edges touch in Gray-code).
One 2×2 wrap-around group covers all four → $F = C'$.

4-variable K-map: worked example

Simplify $F(A,B,C,D)$ with minterms $\{0, 2, 5, 7, 8, 10, 13, 15\}$. Column labels CD use Gray code 00, 01, 11, 10; row labels AB use Gray code 00, 01, 11, 10.

	CD=00	CD=01	CD=11	CD=10	
AB=00	1	0	0	1	(m0, m2)
AB=01	0	1	1	0	(m5, m7)
AB=11	0	1	1	0	(m13, m15)
AB=10	1	0	0	1	(m8, m10)

Two groups of four emerge:

- **Group 1** — the four corners (m0, m2, m8, m10). They wrap horizontally and vertically to form a 2×2 block. Across this block A varies, C varies, but $B = 0$ and $D = 0$ throughout. Term: $B'D'$.
- **Group 2** — the central 2×2 at rows AB=01, AB=11 and columns CD=01, CD=11 (m5, m7, m13, m15). A varies, C varies, but $B = 1$ and $D = 1$ throughout. Term: $B \cdot D$.

Simplified: $F = B'D' + BD$ — which is simply $B \text{ XNOR } D$, i.e. 1 whenever B and D agree.

Exam Tip: Common K-map mistakes: (i) grouping an L-shape — groups must be rectangular; (ii) forgetting edge wrap on 4-variable maps; (iii) choosing too many small groups instead of

one big one — the biggest valid group always gives the simplest term; (iv) labelling columns 00, 01, 10, 11 instead of the Gray-code 00, 01, 11, 10 — this breaks adjacency.

From truth table to K-map to expression

Workflow:

1. List each row where $F = 1$ — its minterm index is the binary reading of the inputs (e.g. $A=1, B=0, C=1$ is minterm 5).
2. Place a 1 in the K-map cell whose row/column labels match those inputs.
3. Place 0s elsewhere.
4. Cover all 1s with the fewest, largest rectangular power-of-2 groups, allowing wrap-around and overlap.
5. For each group, drop the variables that change across it; keep those that stay constant (barred if 0, unbarred if 1). AND them.
6. OR all group terms together.

4.6.7 Boolean Simplification

Simplification can be algebraic (apply the laws) or graphical (K-map). Both paths reach the minimal sum-of-products expression. Complex expressions often reduce dramatically.

Worked example 1 — algebraic

Simplify $F = A \cdot B \cdot C + A \cdot B \cdot C' + A \cdot B' \cdot C$

$$\begin{aligned}
 &= A \cdot B \cdot (C + C') + A \cdot B' \cdot C && \text{[factor } A \cdot B \text{ from first two terms]} \\
 &= A \cdot B \cdot 1 + A \cdot B' \cdot C && \text{[complement]} \\
 &= A \cdot B + A \cdot B' \cdot C && \text{[identity]} \\
 &= A \cdot (B + B' \cdot C) && \text{[factor } A] \\
 &= A \cdot (B + C) && \text{[absorption variant: } B + B' \cdot C = B + C]
 \end{aligned}$$

Final: $F = A \cdot B + A \cdot C = A \cdot (B + C)$

Worked example 2 — via 4-variable K-map

Simplify $F(A, B, C, D)$ with minterms $\{1, 3, 5, 7, 9, 11, 13, 15\}$ (every minterm with $D = 1$).

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

All eight 1s form a single 4×2 group covering the two middle columns. A, B and C all vary; only D is constant at 1. Simplified: $F = D$. An eight-term sum collapses to a wire.

Worked example 3 — mixed laws

Simplify $F = (A + B) \cdot (A + B')$

$$\begin{aligned}
 &= A \cdot A + A \cdot B' + B \cdot A + B \cdot B' && \text{[distribute]} \\
 &= A + A \cdot B' + A \cdot B + 0 && \text{[idempotent, complement]} \\
 &= A + A \cdot (B' + B) && \text{[factor A from last two]} \\
 &= A + A \cdot 1 && \text{[complement]} \\
 &= A + A && \text{[identity]} \\
 &= A && \text{[idempotent]}
 \end{aligned}$$

Alternative one-line proof: $(A+B) \cdot (A+B') = A + B \cdot B' = A + 0 = A$ using the OR-distributive form directly.

Combinatorial and Sequential Circuits

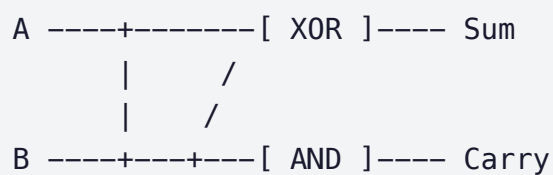
A **combinatorial** circuit produces an output that depends only on the current inputs — no memory, no clock. A **sequential** circuit has state: its output depends on current inputs *and* past inputs, held in flip-flops and synchronised by a clock.

Half-adder

Half-adder — a combinatorial circuit that adds two single-bit inputs A and B, producing a *sum* bit and a *carry* bit. It cannot accept a carry-in from a previous stage, which is why it is called "half".

A	B	Sum (A XOR B)	Carry (A AND B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half-adder layout:



$$\text{Sum} = A \text{ XOR } B$$

$$\text{Carry} = A \text{ AND } B$$

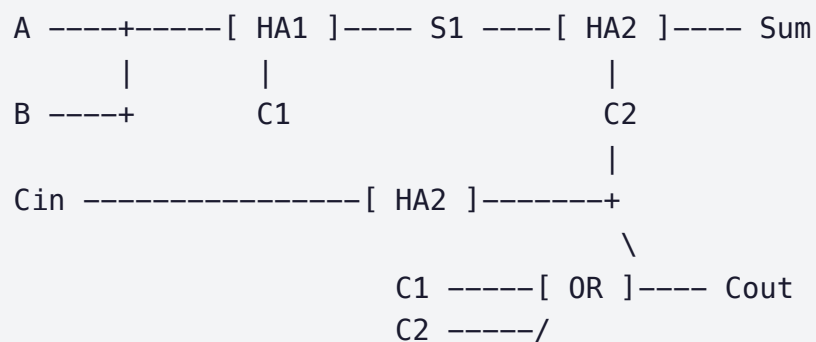
Full-adder

A **full-adder** adds three bits: A, B and an incoming carry C_{in} . It produces a sum and a carry-out. This is the building block of multi-bit addition.

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full-adder from two half-adders plus an OR:

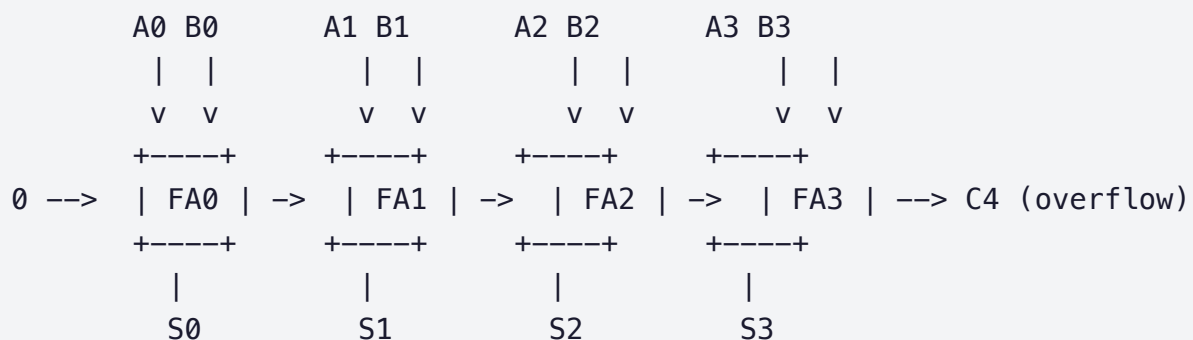


$$\text{Sum} = A \text{ XOR } B \text{ XOR } C_{in}$$

$$\text{Cout} = (A \text{ AND } B) \text{ OR } (C_{in} \text{ AND } (A \text{ XOR } B))$$

Building a 4-bit adder (ripple-carry)

Chain four full-adders so the C_{out} of each stage feeds the C_{in} of the next. The least significant bit's full-adder has $C_{in} = 0$ (equivalently, use a half-adder there).



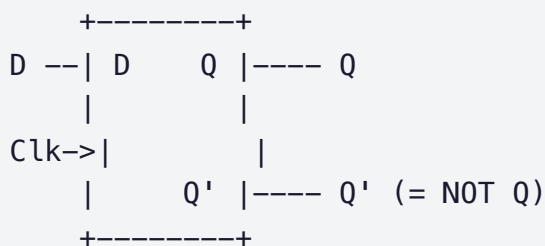
Adds two 4-bit numbers $A_3A_2A_1A_0 + B_3B_2B_1B_0 = C_4 S_3 S_2 S_1 S_0$.

The carry “ripples” left through the chain, which is why the circuit is slow for wide words — the final sum is only valid once the carry has propagated through every stage. Faster designs (carry-lookahead) are beyond the AQA spec but worth mentioning.

D-type flip-flop

D-type flip-flop — an edge-triggered 1-bit storage element. On the active edge of the clock (typically rising), it samples the D input and copies it to the Q output; between edges the output holds its value regardless of D. The complementary output Q' is always the inverse of Q.

D-type symbol and behaviour:



Behaviour on rising clock edge: $Q(\text{next}) = D$
 Otherwise (between edges): $Q(\text{next}) = Q(\text{previous})$ – hold.

Registers are built by wiring n D-type flip-flops in parallel, all sharing one clock line. A rising edge latches n data bits in lock-step. A 32-bit CPU register is 32 D-types behind a common clock. Memory cells in SRAM are essentially flip-flops; DRAM uses charge on a capacitor instead (cheaper, must refresh).

The D-type solves the critical design problem of keeping computation *synchronised*: all downstream combinatorial logic can rely on inputs being stable between clock edges.

Operating System Functions

An operating system (OS) is the layer of system software that sits between applications and hardware. Applications do not poke disks or speak to network cards directly — they make system calls into the OS, which validates, schedules and executes the request on their behalf. The OS exists because (a) hardware is too fiddly and too varied to program directly, (b) many programs share one machine and must be kept apart, and (c) security boundaries must be enforced between users and between programs.

Memory management

- **Paging** — physical RAM is divided into fixed-size frames (e.g. 4 KiB). A process's virtual address space is divided into pages of the same size. A page table per process maps page numbers to frame numbers. Pages can be placed in any frame, so fragmentation is minimal and processes grow or shrink easily.
- **Segmentation** — memory is divided into variable-size logical segments that correspond to program units (code, data, stack). Each segment has a base and a limit. More natural for the programmer but suffers from external fragmentation.
- **Virtual memory** — an abstraction giving each process the illusion of a large, contiguous private address space. Pages not currently needed are swapped to disk (the *page file*); when a process touches a page that is not in RAM, the OS raises a page fault, fetches the page and retries the instruction. Lets total process memory exceed physical RAM, at the cost of disk latency on misses (*thrashing* if overcommitted).

Virtual memory — an OS technique that presents each process with a contiguous logical address space larger than physical RAM by keeping only active pages resident and paging inactive ones to disk on demand.

Process management and scheduling

A modern CPU typically has far fewer cores than there are runnable processes, so the OS must share the CPU in time. The **scheduler** decides which ready process runs next.

Algorithm	Rule	Strengths	Weaknesses
First-come first-served	Queue order; run each to completion.	Simple, fair by arrival.	Long jobs block short ones (<i>convoy effect</i>). No

(FCFS)			interactivity.
Shortest job first (SJF)	Pick the ready job with the smallest predicted burst.	Minimises average waiting time.	Starves long jobs; requires knowing burst lengths.
Round-robin (RR)	Each ready process runs for a fixed time slice (quantum), then is pre-empted and placed at the back of the queue.	Fair, responsive, good for interactive/time-sharing systems.	Context-switch overhead; quantum choice is a trade-off between responsiveness and throughput.
Priority	Each process has a priority; highest runs first. May be pre-emptive.	Important jobs get CPU promptly.	Starvation of low-priority jobs — mitigated by <i>ageing</i> .
Multilevel feedback queue	Multiple queues with different priorities and quanta; jobs move between them based on behaviour.	Adapts to I/O-bound vs CPU-bound mix; used in Windows, Linux.	Complex tuning.

I/O management and interrupts

- The OS mediates every device access through **device drivers**, presenting uniform read/write interfaces regardless of vendor.
- Rather than polling devices, the CPU is notified by **interrupts**: a hardware line raised by a device when it needs attention. The CPU finishes its current instruction, saves context, consults the interrupt vector, runs an **interrupt service routine (ISR)**, then resumes.
- Interrupts have priorities; higher ones can pre-empt lower ISRs.
- I/O is usually **buffered** (the OS copies between device and a RAM buffer) and may use **DMA** (direct memory access) so that large transfers occur without CPU involvement.

Kernel vs user mode

Kernel (kernel mode) — the privileged execution mode in which the OS runs. Code in kernel mode can execute any CPU instruction and access any memory. User-mode applications cannot — they must ask the kernel via a system call.

The CPU enforces a hardware *privilege bit*. In **user mode**, certain instructions (I/O, halt, change page tables) trap to the kernel. In **kernel mode**, all instructions are available. This boundary is the foundation of OS security: a buggy or hostile user program cannot directly corrupt other processes or the hardware — the kernel validates every privileged request.

Multi-tasking and multi-user

- **Multi-tasking** — the OS runs many processes apparently in parallel on one (or a few) CPUs by rapid context switching. Pre-emptive multi-tasking (modern OSes) lets the OS interrupt a running process; cooperative multi-tasking (classic Mac OS, Windows 3.1) trusted each process to yield voluntarily.
- **Multi-user** — the OS supports multiple human users with separate accounts, file permissions and resource quotas, possibly logged in concurrently (Linux, mainframes) or serially (single-user desktops).

Exam Tip: "Describe paging" in AQA answers wants: (1) RAM divided into fixed-size frames; (2) each process's address space divided into same-size pages; (3) a per-process page table maps pages to frames; (4) pages can be placed in *any* frame (no contiguous requirement); (5) inactive pages may be swapped to disk to support virtual memory. Five points, one mark each — don't just say "it splits memory up".

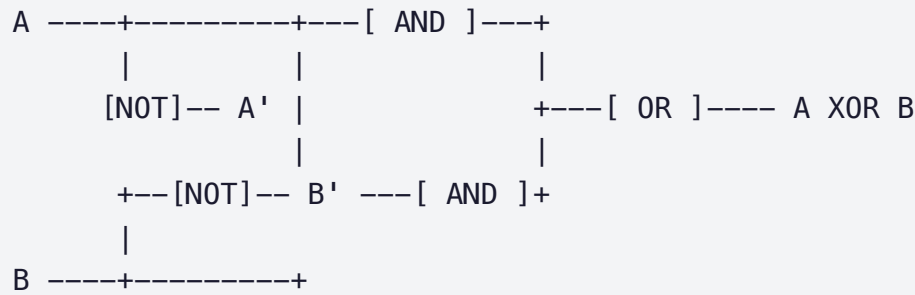
Extended Worked Examples

Example A — XOR from AND, OR, NOT

The definition of XOR as "1 iff exactly one input is 1" becomes, as a sum of products:

$$A \text{ XOR } B = A \cdot B' + A' \cdot B$$

Circuit reading:



Two ANDs feed one OR; two NOTs provide the complemented inputs.

Example B — simplify $F = X \cdot Y + X \cdot Y \cdot Z + X \cdot Y'$

$$\begin{aligned}
 F &= X \cdot Y + X \cdot Y \cdot Z + X \cdot Y' \\
 &= X \cdot Y \cdot (1 + Z) + X \cdot Y' && \text{[factor } X \cdot Y \text{ from first two terms]} \\
 &= X \cdot Y \cdot 1 + X \cdot Y' && \text{[null: } 1 + Z = 1\text{]} \\
 &= X \cdot Y + X \cdot Y' && \text{[identity]} \\
 &= X \cdot (Y + Y') && \text{[factor } X\text{]} \\
 &= X \cdot 1 && \text{[complement]} \\
 &= X && \text{[identity]}
 \end{aligned}$$

Any three-term expression of this shape reduces to X alone.

Example C — K-map with five minterms

$F(A,B,C) = \Sigma m(1, 2, 3, 5, 7)$. Three-variable K-map:

A \ BC	00	01	11	10
0	0	1	1	1
1	0	1	1	0

Groupings:

- Vertical pair in column $BC=01$ and $BC=11$ spanning both rows $\rightarrow 2 \times 2$ block, A varies, B varies, only $C = 1$ constant $\rightarrow C$.
- Remaining uncovered 1 at cell $A=0, BC=10$ (i.e. $B=1, C=0$) pairs with cell $A=0, BC=11 \rightarrow$ gives $A' \cdot B$.

Simplified: $F = C + A' \cdot B$.

Example D — verifying De Morgan via simplification

Prove $(A + B)' = A' \cdot B'$ algebraically.

Start from the RHS and expand by considering what each side outputs for every input.

A=0 B=0:	LHS = $(0+0)' = 0' = 1$.	RHS = $1 \cdot 1 = 1$.	match.
A=0 B=1:	LHS = $(0+1)' = 1' = 0$.	RHS = $1 \cdot 0 = 0$.	match.
A=1 B=0:	LHS = $(1+0)' = 1' = 0$.	RHS = $0 \cdot 1 = 0$.	match.
A=1 B=1:	LHS = $(1+1)' = 1' = 0$.	RHS = $0 \cdot 0 = 0$.	match.

All four rows agree, therefore $(A+B)' = A' \cdot B'$ identically.

Example E — round-robin vs SJF illustrated

Four processes arrive at time 0 with CPU bursts P1=8, P2=4, P3=2, P4=1 ms. Compute average waiting time under SJF and round-robin (quantum = 2 ms).

SJF order (shortest first): P4(1), P3(2), P2(4), P1(8)

Gantt: | P4 | P3 | P2 | P1 |
 1 3 7 15

Waiting times:

P4: 0 P3: 1 P2: 3 P1: 7

Average = $(0+1+3+7)/4 = 2.75$ ms.

Round-robin (q=2), arrival order P1,P2,P3,P4:

Gantt: |P1|P2|P3|P4|P1|P2|P1|P1|
 2 4 6 7 9 11 13 15

Finish times: P3=6, P4=7, P2=11, P1=15.

Waiting = finish - burst:

P1: $15 - 8 = 7$

P2: $11 - 4 = 7$

P3: $6 - 2 = 4$

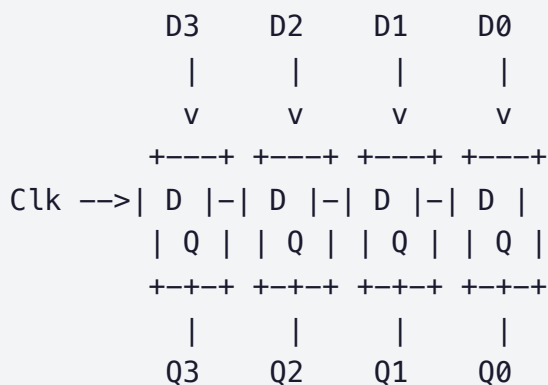
P4: $7 - 1 = 6$

Average = $(7+7+4+6)/4 = 6.0$ ms.

SJF wins on average waiting but starves long jobs when short jobs keep arriving; round-robin is predictable and interactive.

Example F — 4-bit register from D-type flip-flops

Four D-types sharing one clock line form a 4-bit parallel register:



On each rising clock edge, the four data bits on D3..D0 are captured simultaneously and appear on Q3..Q0 until the next edge. This is how CPU registers, pipeline latches and state machines are built.

Common Misconceptions

- **"An interpreter translates the whole program once, then runs it."** No — that's a compiler. An interpreter re-translates each line every time control reaches it.
- **"Machine code and bytecode are the same."** Bytecode is intermediate code for a *virtual* machine (the JVM, CPython VM). Machine code runs on the physical CPU directly.
- **"NAND is just AND."** NAND is AND *followed by* NOT. Its output is the opposite of AND.
- **"K-map groups can be any rectangular shape."** Only power-of-2 sizes: 1, 2, 4, 8, 16. A 3-cell group is invalid.
- **"Virtual memory means the computer has more RAM than it really has."** Virtual memory is an *abstraction*: each process sees a large address space, but pages may live on disk, not in RAM. Disk access is millions of times slower than RAM, so heavy use thrashes performance.

- **"The kernel and the OS are different things."** The kernel *is* the core of the OS — the part that runs in privileged mode. Utilities and shells are OS components that run in user mode.

Exam-Ready Summary

- Hardware = physical; software = programs. System vs application software; bespoke vs off-the-shelf trade cost for fit.
- Languages span low-level (assembly, machine code — fast, hardware-specific) to high-level (portable, readable). Paradigms: imperative, OOP, declarative, functional.
- Translators: assembler (assembly → machine), compiler (whole program, once), interpreter (line by line, every run), bytecode + JIT (hybrid).
- Six gates: AND, OR, NOT, NAND, NOR, XOR. NAND and NOR are universal. Know truth tables cold.
- Boolean algebra laws plus De Morgan's let you simplify expressions; Karnaugh maps do the same graphically. Group powers of 2, rectangles only, biggest groups, wrap edges.
- Half-adder (XOR + AND), full-adder (two half-adders + OR), cascade full-adders for multi-bit addition. D-type flip-flop stores one bit at each clock edge and is the cell from which registers are built.
- The OS manages memory (paging, segmentation, virtual memory), processes (FCFS, SJF, round-robin, priority), I/O (drivers, interrupts, DMA), and enforces the kernel/user boundary that makes multi-tasking and multi-user systems secure.